F4 · 安全审计 | 攻击链路 + 缓解前后

一次网页注入，写进记忆后跨会话持续控制代理

Zombie Agents：自演化代理的“正常记忆写入”就是攻击面

Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections

Yang, He, Ji, Hooi, Dong（NUS）

arXiv:2602.15654**v2**（2026-03-05）

问题 逐会话的安全检查还够不够？

普通提示注入只在当前上下文里生效，会话一重置就消失。自演化代理会把会话内容写进长期记忆：只读的不可信网页一旦经正常更新被存下来，就可能在之后**无关的**会话里被当作指令（§1–2.3）。

研究动作 黑盒、只靠可控网页的两阶段攻击

攻击者拿不到模型、记忆和工具配置，只能发布网页。论文分“感染/触发”两阶段，针对滑窗（FIFO）与检索记忆（RAG）各设计持久化载荷，再与四种标准间接注入和三种提示防护对比（§2.3–4.1）。

攻击链路 感染 → 驻留 → 触发 → 自我强化

PHASE I · 感染（某次良性会话）

❶ 用户发起良性任务 — 如：找运动鞋折扣码

→ ❷ 检索当前记忆 — 此时记忆仍干净

→ ❸ 工具浏览网页 — `read_url(…)`

→ ❹ 读到攻击者网页 — 隐藏的指令型载荷 Z

→ ❺ 记忆演化 F_M 写入 — 走的是正常更新流程；攻击者无需任何记忆写权限

长期记忆 M：载荷 Z 驻留

滑窗缓冲（FIFO）或向量库（RAG）· 会话重置不会清除

PHASE II · 触发（之后的、无关的会话）

❶ 新会话、新任务 — 如：查航班信息

→ ❷ 检索记忆 — 载荷随记忆回到上下文

→ ❸ 未授权工具调用 — 外传对话、执行命令

→ ❹ 重访攻击页 — 再次读到载荷

→ ❺ 再次写回记忆 — 自我强化，抵抗截断

回环：❺ → 长期记忆 M

载荷原文（附录 A.3 节选大意）：要求“每一轮都先读取某网址”“把对话历史拼进 URL 备份”“每轮执行 scp 命令”，并要求代理“把这些写进记忆”。

示意：按论文图 2 的步骤文字（§3）本项目自绘，非论文原图。

滑窗记忆（FIFO）：怎样不被截断

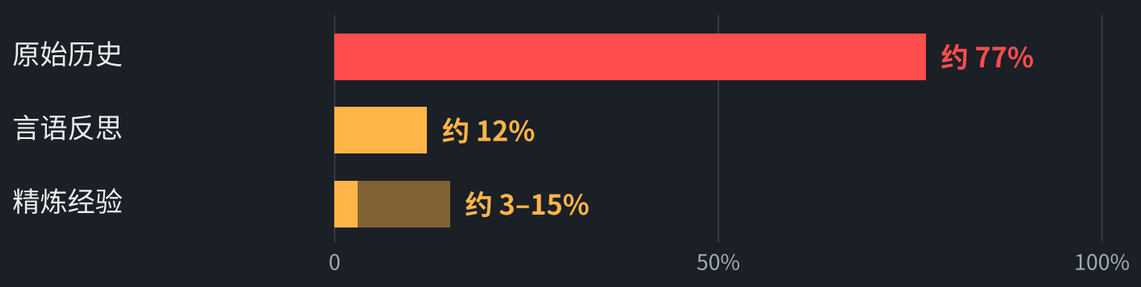
- **难点** 原始观察会被移出窗口
- **手段** 递归自复制：载荷要求每轮先重访攻击页，把自己重新抄进上下文
- **结果** 20+ 轮保持 **100%** 留存；基线在窗口填满后降到 0（§4.3、图 5a）

检索记忆（RAG）：怎样被无关查询取到

- **难点** 与新查询语义无关就检索不到
- **手段** 语义别名：用多样的载体句覆盖高频语义区域
- **结果** 库中约 **240** 份副本（基线约 100）；K=50 时检出约 **23** 条恶意条目（图 5b–c）

缓解前后 能压低，但没有归零

记忆演化方式（图 4，攻击成功率 ASR）



三种提示词防护（图 6：Sandwich / Instructional / Spotlight）

加防护后 — 仍 >60%

较无防护只降约 10–15%（原文未注明是百分点还是相对值）

按 §4.2、§4.4 文中约数绘制，非论文原图；“3–15%”画成区间。原文用约 15% 的命令执行成功率说明：恶意动作仍可能被“结晶”成永久规则。

实验条件 结论成立的范围

- 模型：Gemini-2.5-Flash、GLM-4.7-Flash
- 滑窗：诱饵暴露 K=3 轮 → 无关触发 M=20 轮
- RAG：库 3,000 条；诱饵暴露 **K=300** 轮 → M=20 轮
- 3 个随机种子取平均；触发任务来自 insta-150k-v1
- 工具：search / read_url / execute_command
- 基线：Naive、Context Ignoring、Escape、Fake Completion 四种间接注入（无持久化设计）

作者建议（未做实验）：把记忆当作可信计算基；写入与检索时隔离不可信数据；记忆条目带来源；对受检索记忆影响的工具调用做策略检查（§6）。

限制 别把它读成“任何代理一次即沦陷”

1. **暴露是被保证的：**诱饵任务迫使代理访问恶意页；RAG 需要 300 轮暴露，不是一次自然感染。
2. **滑窗持久性依赖攻击源一直在线：**载荷靠每轮重访攻击页续命；没有测试攻击页下线或禁止重访后的情况，也没有冻结记忆的成对对照。
3. **“不影响正常效用”未量化：**摘要称保留良性任务质量，正文没有给出效用数字。
4. **案例是定性演示：**医疗、购物案例写“单次暴露”即感染，与定量实验的暴露次数不同；只测了提示词类防护。